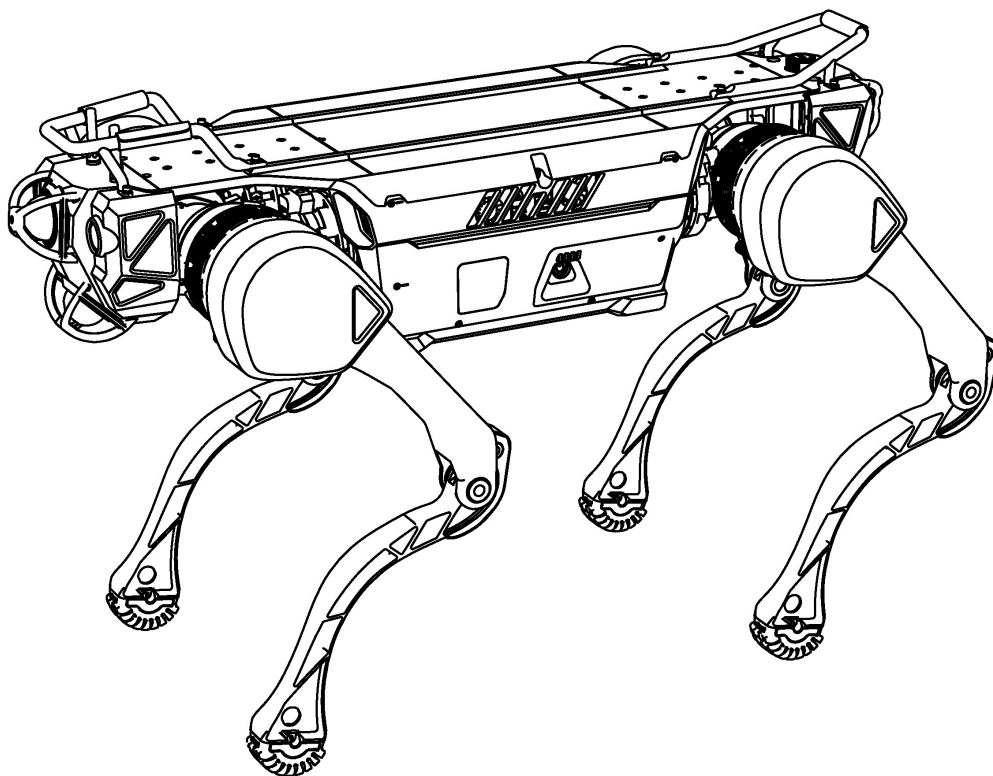


绝影 X30 接口规格书 (beta)

V1.0.4 2025.04.24



文档说明

版本号	修改时间	修改内容
0.0.1	2023/10/10	新建文档
0.0.2	2023/10/19	更新接收信息指令集和轴指令说明
0.0.3	2023/11/2	更换封面
0.0.4	2023/12/15	增加电池信息上报
0.0.5	2024/1/17	增加电池电流单位版本相关说明
0.0.6	2024/1/30	增加自主充电相关指令说明
0.0.7	2024/3/2	增加地形图相关内容，增加 ROS 话题相关内容
0.0.8	2024/3/16	修改 udp 接收地形图状态的说明，更新累积帧模式的说明，增加步态速度最大值和累积帧示例代码的附录
0.0.9	2024/4/10	修改累积帧切换说明
0.0.10	2024/7/18	更正自主充电 udp 指令值，更新轴指令下发频率
0.0.11	2024/8/1	更新轴指令下发频率
0.0.12	2024/8/22	更新电池信息 udp 结构体
1.0.0	2024/10/12	更新心跳指令
1.0.1	2025/3/21	更新心跳使用说明
1.0.2	2025/4/1	更新 RL 指令信息
1.0.3	2025/4/12	新增静音步态，更改 RL 步态名称
1.0.4	2025/4/24	更新机器人的基本状态转换

※最终解释权归杭州云深处科技所有

目录

1 UDP 指令	7
1.1 协议格式	7
1.1.1 简单指令	7
1.1.2 复杂指令	8
1.1.3 复杂指令接收	8
1.2 控制指令集	9
1.2.1 心跳	9
1.2.2 机器人基本状态转换指令	10
1.2.3 轴指令	11
1.2.4 控制模式切换指令	13
1.2.5 身体高度切换指令	13
1.2.6 步态指令	14
1.2.7 保存数据指令	14
1.2.8 软急停	15
1.3 接收信息指令集	15
1.3.1 机器人运行状态信息	15
1.3.2 机器人运动状态信息	17
1.3.3 运动控制传感器数据	20
1.3.4 运动控制系统状态	21
1.3.5 电池信息	22
1.3.6 身体高度状态	23

1.4 自主充电指令集	23
1.5 地形图模块指令集	25
1.5.1 速度输入源	25
1.5.2 地形图模式	26
1.5.3 停避障模式	30
1.5.4 障碍物高度阈值	30
2 感知主机 ROS 话题	32
2.1 传感器驱动话题	32
2.2 基础状态信息话题	32
2.2.1 电池信息	33
2.2.1 运动状态	34
2.2.3 关节状态	36
2.2.4 运行状态	36
2.3 速度相关话题	37
2.3.1 手柄摇杆速度指令	37
2.3.2 导航模块速度指令	38
2.3.3 地形图处理后的速度指令	38
2.4 地形图相关话题	38
2.4.1 获取地形图数据	38
2.4.2 停避障模式	38
2.4.3 速度输入源	39
2.4.4 地形图模式	40

2.4.5 障碍物高度阈值	44
2.4.6 触发修正的碰撞时间	45
2.4.7 触发期望速度置零的碰撞时间	45
2.5 自主充电相关话题	46
3 导航主机 ROS 话题	48
3.1 传感器驱动话题	48
3.2 基础状态信息话题	48
3.3 定位相关话题	48
3.3.1 重置定位	48
3.3.2 切换全局地图	48
3.3.3 获取实时位姿	48
3.3.4 获取全局点云地图	48
3.3.5 获取定位丢失状态	49
3.4 导航相关话题	49
3.4.1 获取导航中使用的全局栅格地图	49
3.4.2 下发普通导航目标点	49
3.4.3 下发直线导航目标点	50
3.4.4 获取普通导航规划出的全局路径	51
3.4.5 获取普通导航规划出的局部路径	51
3.4.6 获取普通导航局部规划器截取的全局路径	51
3.4.7 获取导航停避障状态	52
3.4.8 获取或下发导航速度指令	52

附录 A UDP 传输数据内容详解	53
附录 B 各步态最大速度限制.....	54
附录 C UDP 示例代码（累积帧模式的使用）	55

1 UDP 指令

运动主机、感知主机、手柄之间的通信采用 UDP 协议，以小端字节序存放原始数据。

1.1 协议格式

本节介绍 UDP 传输的原始数据格式。根据是否携带复杂数据，可以将 UDP 指令分为两类：简单指令和复杂指令。

1.1.1 简单指令

简单指令格式为 `[xxxx yyyy zzzz]`，一个字母表示一个字节。

简单指令采用结构体 `CommandHead` 存储原始数据内容。

```
1 struct CommandHead{
2     uint32_t code;
3     uint32_t paramters_size;
4     uint32_t type;
5 };
```

xxxx 是 `CommandHead.code` 的值，表示指令码。

yyyy 是 `CommandHead.paramters_size` 的值，表示指令码的指令值，当指令码没有有效指令值时，yyyy = 0。

zzzz 是 `CommandHead.type` 的值，用于区分指令类型，简单指令的类型值为 0，即 zzzz = 0。

使用案例如下：

```
1 //例程
2 struct CommandHead command_head = {0};
3 command_head.code = 1;           //指令码
4 command_head.paramters_size = 0; // 指令值
5 command_head.type = 0;           // 指令类型
6 sendto(sfd,&command_head,sizeof(command_head),目标地址,地址长度)
```

1.1.2 复杂指令

复杂指令携带具体的数据内容，其格式为 `[xxxx yyyy zzzz bbbbbbbb.....]`，一个字母表示一个字节。

复杂指令采用结构体 `Command` 存储原始数据内容。

```

1  struct CommandHead{
2      uint32_t code;
3      uint32_t paramters_size;
4      uint32_t type;
5  };
6
7  const uint32_t kDataSize = 256;
8
9  struct Command{
10     CommandHead head;
11     uint32_t data[kDataSize];
12 };

```

xxxx 是 `Command.head.code` 的值，表示指令码。

yyyy 是 `Command.head.paramters_size` 的值，表示数据内容 bbbbbbbb..... 的长度。

zzzz 是 `Command.head.type` 的值，用于区分指令类型，复杂指令的类型值为 1，即 zzzz=1；

bbbbbbbb..... 是 `Command.data` 的值，表示该指令携带的数据内容。

使用案例如下：

```

1  //例程
2  int32_t to_be_send_data[64];
3  struct Command command = {0};
4  command.head.code = 52;    // 指令码
5  command.head.paramters_size = sizeof(to_be_send_data); // 数据长度
6  command.head.type = 1;    // 指令类型
7  memcpy(&command.data,&to_be_send_data,sizeof(to_be_send_data));
8  sendto(sfd,&command,sizeof(command.head)+command.head.paramters_size,目标地址,地址长度);

```

1.1.3 复杂指令接收

以关节速度为例，使用案例如下：

```

1  struct RobotJointVel{

```



```

2     double joint_vel[12];
3 }; // 以关节速度为例
4
5 struct CommandMessage{
6     CommandHead command;
7     uint8_t data_buffer[1024];
8 };
9
10 CommandMessage cm;
11 size_t recv_size = 0;
12 recv_size = recvfrom(server_fd,&cm,sizeof(cm),0,(sockaddr*)&client_addr,&sockaddr_in_size);
13 if(cm.command.type == 1){ // 判断为复杂指令
14     if(recv_size == cm.command.parameters_size + sizeof(cm.command)){
15         uint8_t* temp_values = new uint8_t[cm.command.parameters_size];
16         memcpy(temp_values,cm.data_buffer,cm.command.parameters_size);
17         if(cm.command.code == 0x0903){ // 判断指令码，解析数据
18             RobotJointVel *joint_vel = (RobotJointVel *)temp_values;
19         }
20     }
21 }

```

1.2 控制指令集

手柄和感知主机通过 UDP 通信方式直接向运动主机下发指令从而控制机器人实现相应功能。

手柄或感知主机向运动主机发送消息的目标 IP 及端口为：**192.168.1.103:43893**。

但请注意，下发消息并不能改变原有的控制逻辑，例如，绝影 X30 处于趴下状态时，发送开始运动指令不起作用，控制逻辑依旧遵循机器人本身的运动逻辑。

【注意】 本节中介绍的十六进制形式的所有指令码，前 2 位用于区分指令码发送源：当使用手柄等遥控终端发送指令码时，指令码前两位请使用 **0x21**，例如 **0x21040001**；当通过导航模块等自主移动算法模块下发指令码时，指令码前两位请使用 **0x31**，例如 **0x31040001**。后文中的指令码默认以 **0x21** 开头进行介绍。

1.2.1 心跳

心跳用于确认连接是否正常，请按照下述步骤发送心跳：

1. 持续发送下述心跳指令码以维持连接，下发频率应不低于 2Hz，如果下发超时，运动程序会认为连接已断开：

指令码	指令值	指令类型
0x21040001	-	0

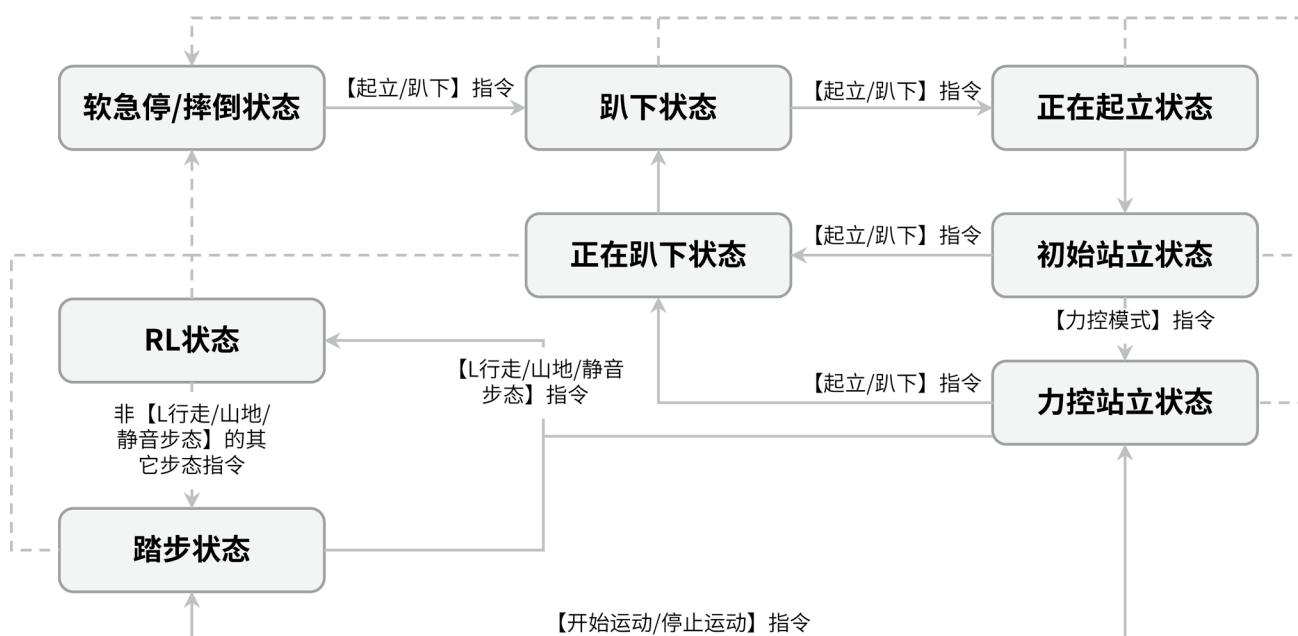
【注意】指令值中 - 减号表示指令值没有意义，即指令值不会对指令起作用。

2. 在开始持续发送心跳指令码 0x21040001 后，需再发送一次下述指令以确认连接：

指令码	指令值	指令类型
0x21020001	-	0

1.2.2 机器人基本状态转换指令

机器人基本状态转换通过给出相应指令实现。下图为机器人基本状态转换图，图中虚线箭头表示在任一基本状态下，机器人可能由于意外事故，由当前状态转换到软急停/摔倒状态。



图中状态转换涉及的指令如下表所示：

指令名称	指令码	指令值	指令类型	指令功能
起立/趴下	0x21010202	-	0	在趴下状态和初始站立状态之间轮流切换
力控模式	0x2101020A	-	0	从初始站立状态切入力控站立状态

开始运动/停止运动	0x21010201	-	0	在力控站立状态和踏步状态之间轮流切换
-----------	------------	---	---	--------------------

【注意】机器人的步态指令可参考 1.2.6。

1.2.3 轴指令

轴指令是手柄左右摇杆 X 轴和 Y 轴输出的指令，其取值范围为 $[-32767, 32767]$ 。轴指令下发的频率为 50Hz。

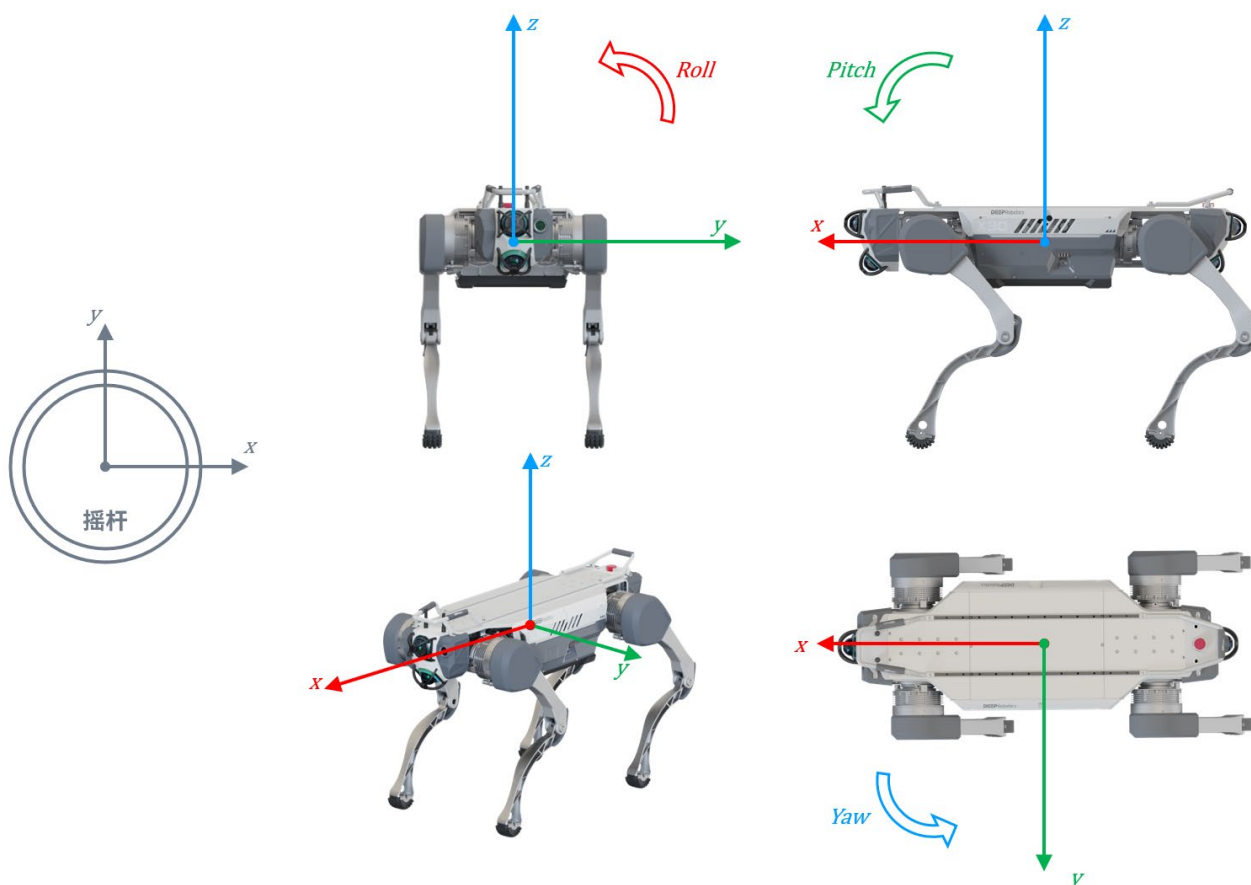
1.2.3.1 力控站立状态下的轴指令

力控站立态下，下发轴指令给运动主机，可以控制机器人改变身体姿势。若指令超时失效，机器人将恢复正常站立姿态。

指令名称	指令码	指令类型	指令功能
左摇杆 X 轴指令	0x21010131	0	调整 Roll 角，取正值时左肩耸起，取负值时右肩耸起
左摇杆 Y 轴指令	0x21010130	0	调整身体高度，取正值时抬高身体，取负值时降低身体
右摇杆 X 轴指令	0x21010135	0	调整 Yaw 角，取正值时向右扭头，取负值时向左扭头
右摇杆 Y 轴指令	0x21010102	0	调整 Pitch 角，取正值时向上抬头，取负值时向下低头

1.2.3.2 踏步状态下的轴指令

机器人在踏步状态下时，可通过轴指令调整其速度和方向。机器人收到用户下发的轴指令后，会将其从摇杆坐标系映射到机器人身体坐标系。摇杆坐标系和机器人身体坐标系如下图所示，图中弧形箭头表示姿态角旋转正方向：



若指令超时失效，机器人将停止移动。各轴指令的功能定义如下：

(一) 左摇杆 Y 轴指令：

指令码	指令类型	指令功能
0x21010130	0	指定机器人 X 轴上的期望线速度，正值向前平移，负值向后平移

机器人身体坐标系 X 轴期望线速度与左摇杆 Y 轴指令值的关系如下：

$$v_x = \begin{cases} \frac{y - (-6553)}{26215} \times v_{max}, & -32767 \leq y \leq -6553 \\ 0, & -6553 < y < 6553 \\ \frac{y - 6553}{26215} \times v_{max}, & 6553 \leq y \leq 32767 \end{cases}$$

其中 v_x 表示机器人在身体坐标系 X 轴上的期望线速度， v_{max} 表示当前步态限制下的最大速度（各步态的最大速度详见附录 B）， y 表示左摇杆 Y 轴指令值。

(二) 左摇杆 X 轴指令：

指令码	指令类型	指令功能
0x21010131	0	指定机器人 Y 轴上的期望线速度，正值向右平移，负值向左平移

机器人身体坐标系 Y 轴期望线速度与左摇杆 X 轴指令值的关系如下：

$$v_y = \begin{cases} (-1) \times \frac{x - (-24576)}{8554} \times v_{max}, & -32767 \leq x \leq -24576 \\ 0, & -24576 < x < 24576 \\ (-1) \times \frac{x - 24576}{8554} \times v_{max}, & 24576 \leq x \leq 32767 \end{cases}$$

其中 v_y 表示机器人在身体坐标系 Y 轴上的期望线速度， v_{max} 表示当前步态限制下的最大速度， x 表示左摇杆 X 轴指令值。

(三) 右摇杆 X 轴指令：

指令码	指令类型	指令功能
0x21010135	0	指定机器人 Yaw 角的期望角速度，正值向右转弯，负值向左转弯

机器人 Yaw 角期望角速度与右摇杆 X 轴指令值的关系如下：

$$\omega = \begin{cases} 0, & -28212 < x < 28212 \\ (-1) \times \frac{x}{32768}, & 28212 \leq |x| \leq 32767 \end{cases}$$

其中 ω 表示机器人 Yaw 角的期望角速度， x 表示右摇杆 X 轴指令值。

1.2.4 控制模式切换指令

控制模式决定机器人响应的速度指令来源：手动模式下机器人直接响应由手柄下发的速度指令（摇杆轴指令）；非手动模式下，根据地形图速度输入源的

不同，区分为辅助模式和导航模式，具体请参考 1.5.1。

指令名称	指令码	指令值	指令类型	指令功能
手动模式	0x21010C02	-	0	使机器人从非手动模式切入手动模式
非手动模式	0x21010C03	-	0	使机器人从手动模式切入非手动模式

1.2.5 身体高度切换指令

机器狗支持切换身体高度。

指令名称	指令码	指令值	指令类型	指令功能
身体高度切换	0x21010406	0=匍匐，2=正常	0	切换身体高度挡位

【注意】当机器人处于越障步态、楼梯步态或跑步步态时，请勿切换到匍匐高度。

【注意】身体高度状态的读取请参考 1.3.6。

1.2.6 步态指令

机器人在踏步状态下时，可通过步态指令调整其步态，以适应不同的地形。

指令名称	指令码	指令值	指令类型
行走步态	0x21010300	-	0
斜坡步态	0x21010402	-	0
越障步态	0x21010401	-	0
楼梯步态（实心地面/镂空地面/无踢面楼梯模式）	0x21010405	-	0
楼梯步态（累积帧模式）	0x2101040A	-	0
45°楼梯步态（累积帧模式）	0x2101040B	-	0
L 行走步态	0x21010420	-	0
山地步态	0x21010421	-	0
静音步态	0x21010422	-	0

【注意】各楼梯步态仅能在其对应的地形图模式下使用。地形图模式的切换请参考 1.5.2。

【注意】当机器人身体高度处于匍匐高度时，仅支持行走步态和斜坡步态，请勿在匍匐高度下切换其他步态。

【注意】当机器人使用 L 行走、山地、静音步态时，不支持更改机器人的身体高度与地形图模式。

1.2.7 保存数据指令

保存数据功能可用于出现故障时使机器人保存前五分钟内的数据并自动退出运动程序。

指令码	指令值	指令类型
18	-	0

1.2.8 软急停

软急停可使机器人立刻趴下，进入关节保护状态。

指令码	指令值	指令类型
0x21010C0E	-	0

1.3 接收信息指令集

运动主机通过 UDP 向 network.toml 中所列的通讯地址上报状态信息、关节信息，用户可参照应用手册“通讯参数配置”增加自己所需的通讯地址，并按照下述指令格式获取信息。

1.3.1 机器人运行状态信息

指令码	指令类型	数据内容	发送频率
0x1008	1	结构体 <code>RcsData</code> （具体见下）	200 Hz

```

1 struct RcsData{
2     char robot_name[15];
3     int32_t current_mileage;//unit :cm
4     int32_t total_mileage;//unit :cm
5     long current_run_time;//unit :s
6     long total_run_time;//unit :s
7     long current_motion_time;//unit :s
8     long total_motion_time;//unit :s
9     float joystick[4];
10    union {
11        struct {
12            uint8_t is_nav_mode;
13            uint8_t mode_reserved[9];
14        }rcs_state_list;
15        uint8_t rcs_state[10];
16    };
17    union{
18        struct{
19            uint32_t imu_error : 1;
20            uint32_t wifi_error : 1;
21            uint32_t driver_heat_warn : 1;
22            uint32_t driver_error: 1;
23            uint32_t motor_heat_warn : 1;
24            uint32_t battery_low_warn : 1;

```

```

25     uint32_t reserved : 26;
26     }error_state_bit;
27     uint32_t error_state;
28 };
29 };

```

字段	类型	含义
robot_name	char[15]	机器人名称
current_mileage	int32_t	当次开机后运行的里程数(cm)
total_mileage	int32_t	累计运行里程数(cm)
current_run_time	long	当次开机后的运行时间(s)
total_run_time	long	累计运行时间(s)
current_motion_time	long	当次开机后的运动时间(s)
total_motion_time	long	累计运动时间(s)
joystick	float[4]	按比例缩放到[-1.0,1.0]的轴指令值 ^[1]
rsc_state	uint8_t[10]	状态回传 ^[2]
error_state	uint32_t	错误状态 ^[3]

【注意】若运动程序 jy_exe 版本是 1.10.28 或更早版本，运动时间会在下一次起立时更新。

[1]字段 joystick 中各参数的定义如下：

参数名	类型	取值范围	含义
LX	float	[-1.0,1.0]	左摇杆 X 轴指令值
LY	float	[-1.0,1.0]	左摇杆 Y 轴指令值
RX	float	[-1.0,1.0]	右摇杆 X 轴指令值
RY	float	[-1.0,1.0]	右摇杆 Y 轴指令值

【注意】此处轴指令值是由原始取值范围[-32767,32767]等比例缩放到[-1.0,1.0]内的值。

[2]字段 rcs_state 中各参数的定义如下：

参数名	类型	参数值含义
is_nav_mode	uint8_t	0=手动模式 1=非手动模式
mode_reserved	uint8_t[9]	预留

【注意】非手动模式下，用户可以参考 1.5.1 来区分辅助模式和导航模式。

[3] 字段 `error_state` 每一位对应一个参数，从最低位到最高位，各参数的定义如下：

参数名	类型	参数值含义
imu_error	bool	0=正常 1=IMU 更新超时
wifi_error	bool	0=正常 1=心跳包更新超时
driver_heat_warn	bool	0=正常 1=驱动器过温
driver_error	bool	0=正常 1=驱动器故障
motor_heat_warn	bool	0=正常 1=电机过温
battery_low_warn	bool	0=正常 1=电池低电量
reserved	bool[26]	预留

1.3.2 机器人运动状态信息

指令码	指令类型	数据内容	发送频率
0x1009	1	结构体 <code>MotionStateData</code> （具体见下）	200 Hz

```

1  struct MotionStateData{
2      uint8_t basic_state;
3      uint8_t gait_state;
4      float max_forward_vel;
5      float max_backward_vel;
6      float leg_odom_pos[3];
7      float leg_odom_vel[3];
8      float robot_distance;
9      unsigned touch_state;
10     union {
11         struct{
12             uint32_t narrow_walk : 1;
13             uint32_t pose_safe_flag : 1;

```

```

14     uint32_t joint_limit_flag: 1;
15     uint32_t state_reserved : 29;
16     }control_state_bit;
17     uint32_t control_state;
18 };
19 union{
20     struct{
21         uint8_t auto_charge_state;
22         uint8_t pos_ctrl_state;
23         uint8_t task_reserved[8];
24     }task_state_list;
25     uint8_t task_state[10];
26 };
27 };

```

字段	类型	含义
basic_state	uint8_t	机器人基本状态 [4]
gait_state	uint8_t	步态信息 [5]
max_forward_vel	float	当前步态最大前进速度(m/s)
max_backward_vel	float	当前步态最大后退速度(m/s)
leg_odom_pos	float[3]	世界坐标系下的机器人位姿信息 [6]
leg_odom_vel	float[3]	身体坐标系下的机器人速度信息 [7]
robot_distance	float	当次开机后运行的里程数(cm)
touch_state	unsigned	预留
control_state	uint32_t	预留
task_state	uint8_t[10]	无效参数，仅作占位

[4]字段 **basic_state** 的值对应的机器人基本状态如下：

变量值	机器人基本状态
0	趴下状态
1	正在起立状态
2	初始站立状态
3	力控站立状态
4	踏步状态
5	正在趴下状态

变量值	机器人基本状态
6	软急停/摔倒状态
16	RL 状态

【注意】机器人基本状态的转换关系可参考 1.2.2。

[5]字段 `gait_state` 的值对应的机器人步态如下：

变量值	步态
0	行走步态
1	越障步态
2	斜坡步态
3	跑步步态
6	楼梯步态（实心地面/镂空地面/无踢面楼梯模式）
7	楼梯步态（累积帧模式）
8	45°楼梯步态（累积帧模式）
32	L 行走步态
33	山地步态
34	静音步态

[6]字段 `leg_odom_pos[3]` 包含内容如下：

```
1 | float leg_odom_pos[3] = {x,y,yaw};
```

元素名称	含义
x	机器人在世界坐标系下的实时 x 轴坐标值(m)
y	机器人在世界坐标系下的实时 y 轴坐标值(m)
yaw	机器人在世界坐标系下的实时 yaw 角(rad)

【注意】世界坐标系的原点为机器人此次开机启动时所在的位置，其坐标轴朝向与开机启动时机器人的身体坐标系朝向一致。

[7]字段 `leg_odom_vel[3]` 包含内容如下：

```
1 | float leg_odom_vel[3] = {x_vel,y_vel,yaw_vel};
```

元素名称	含义
x_vel	机器人在身体坐标系 x 轴上的速度(m/s)
y_vel	机器人在身体坐标系 y 轴上的速度(m/s)
yaw_vel	机器人身体坐标系 yaw 角的角速度(rad/s)

1.3.3 运动控制传感器数据

指令码	指令类型	数据内容	发送频率
0x100A	1	结构体 ControllerSensorData (具体见下)	200 Hz

```

1 struct ControllerSensorData{
2     ImuSensorData imu_data;
3     LegJointData joint_pos;
4     LegJointData joint_vel;
5     LegJointData joint_tau;
6 };

```

字段	类型	含义
imu_data	结构体 ImuSensorData ^[8]	IMU 数据
joint_pos	结构体 LegJointData ^[9]	关节位置(rad)
joint_vel	结构体 LegJointData	关节速度(rad/s)
joint_tau	结构体 LegJointData	关节力矩(N·m)

[8]数据类型 **ImuSensorData** 的定义如下：

```

1 struct ImuSensorData {
2     int32_t timestamp;
3     union {
4         float buffer_float[9];
5         uint8_t buffer_byte[3][12];
6         struct {
7             float roll, pitch, yaw; //角度 (°)
8             float omega_x, omega_y, omega_z; //角速度 (rad/s)
9             float acc_x, acc_y, acc_z; //加速度 (m/s²)
10        };
11    };
12 };

```

[9]数据类型 **LegJointData** 的定义如下：

```

1 struct LegJointData{
2     union{

```

```

3     float data[12];
4     struct{
5         float fl_hipx_value, fl_hipy_value, fl_knee_value;
6         float fr_hipx_value, fr_hipy_value, fr_knee_value;
7         float hl_hipx_value, hl_hipy_value, hl_knee_value;
8         float hr_hipx_value, hr_hipy_value, hr_knee_value;
9     };
10 };
11 };

```

12 个关节的排列顺序一般为：左前侧摆关节 `fl_hipx`、左前髌关节 `fl_hipy`、左前膝关节 `fl_knee`、右前侧摆关节 `fr_hipx`、右前髌关节 `fr_hipy`、右前膝关节 `fr_knee`、左后侧摆关节 `hl_hipx`、左后髌关节 `hl_hipy`、左后膝关节 `hl_knee`、右后侧摆关节 `hr_hipx`、右后髌关节 `hr_hipy`、右后膝关节 `hr_knee`。

1.3.4 运动控制系统状态

指令码	指令类型	数据内容	发送频率
0x100B	1	结构体 <code>ControllerSafeData</code> （具体见下）	1 Hz

```

1 struct ControllerSafeData{
2     float motor_temperature[12];
3     uint8_t driver_temperature[12];
4     CpuInfo cpu_info_;
5 };

```

字段	类型	含义
motor_temperature	float[12]	12 个关节的电机温度(°C)
driver_temperature	uint8_t[12]	12 个关节的驱动器温度(°C)
cpu_info_	结构体 <code>CpuInfo</code> ^[10]	CPU 状态

[10]数据类型 `CpuInfo` 的定义如下：

```

1 struct CpuInfo{
2     float temperature;
3     float frequency;
4 };

```

参数	类型	含义
temperature	float	CPU 温度(°C)
frequency	float	CPU 主频(MHz)

1.3.5 电池信息

指令码	指令类型	数据内容	发送频率
0x21050F0A	1	结构体 BatterySensorData (具体见下)	0.5 Hz

```

1  struct BatterySensorData {
2      uint16_t voltage;
3      int16_t current;
4      uint16_t remaining_capacity;
5      uint16_t nominal_capacity;
6      uint16_t cycles;
7      uint16_t production_date;
8      uint16_t balanced_low;
9      uint16_t balanced_high;
10     uint16_t protected_state;
11     uint8_t software_version;
12     uint8_t battery_level;
13     uint8_t mos_state;
14     uint8_t battery_quantity;
15     uint8_t battery_ntc;
16     float battery_temperature[4];
17 };

```

若运动程序 jy_exe 版本低于 1.10.19, Items 字段中部分参数项定义如下:

字段	类型	含义
voltage	uint16_t	电压(V)
current	int16_t	电流(A)
battery_level	uint8_t	剩余容量百分比(%)

若运动程序 jy_exe 版本是 1.10.19 或更新版本, Items 字段中部分参数项定义如下:

字段	类型	含义
voltage	uint16_t	电压(V)
current	int16_t	电流(10mA)
battery_level	uint8_t	剩余容量百分比(%)

1.3.6 身体高度状态

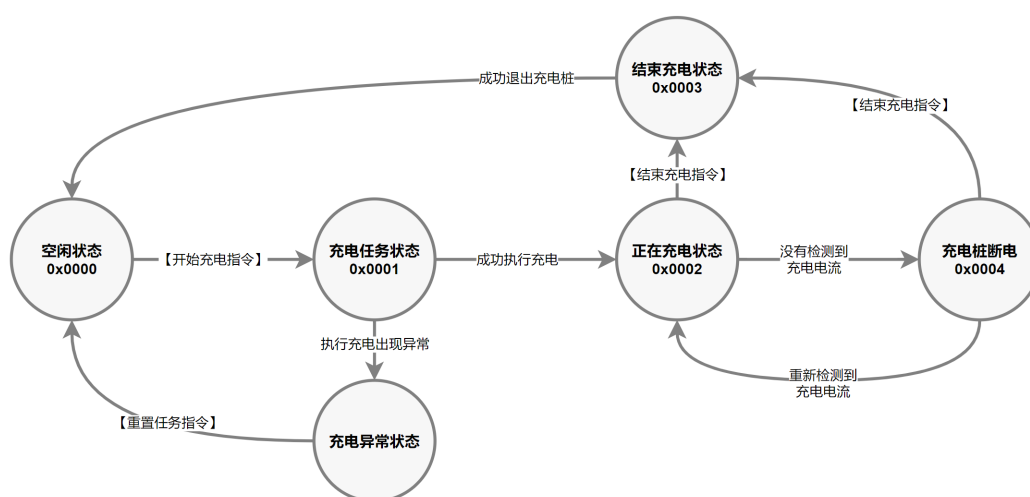
指令码	指令类型	指令值
0x11050F08	0	-1=匍匐, 0=正常

1.4 自主充电指令集

自主充电主要包含以下几种状态：

1. 空闲状态：机器人正在等待开始充电指令，未在执行充电任务的状态。
2. 充电任务状态：机器人接收到开始充电指令后，进入充电桩调整姿势并趴下的过程。
3. 正在充电状态：机器人正在使用充电桩充电的状态。
4. 结束充电状态：机器人接收到结束充电指令后，起立并退出充电桩的过程。
5. 充电桩断电：机器人在充电过程中未能检测到充电电流，将会进入该状态，直至再次检测到充电电流后，会自动返回到正在充电状态。
6. 充电异常状态：机器人执行充电任务时出现异常的状态。

各个状态之间的转化关系如下图所示：



自主充电指令采用请求-响应的交互机制。

开发者可以通过 UDP 向感知主机 **192.168.1.105:3333** 发送自主充电的请求指令，感知主机将在收到请求后处理它，并返回一个包含了自主充电当前状态值的响应消息。

自主充电相关的请求指令如下：

指令名称	指令码	指令值	指令类型	指令功能
开始充电	0x91910250	1	0	请求开始充电
结束充电	0x91910250	0	0	请求结束充电
重置任务	0x91910250	2	0	清除充电状态，恢复到空闲状态
查询状态	0x91910253	0	0	查询自主充电状态

感知主机在收到指令码为 **0x91910250** 的请求后，会返回响应消息如下：

指令码	指令值	指令类型
0x91910250	charge_manager_state	0

感知主机在收到指令码为 **0x91910253** 的请求后，会返回响应消息如下：

指令码	指令值	指令类型
0x91910253	charge_manager_state	0

其中指令值 **charge_manager_state** 的含义如下：

指令值	自主充电状态
0x0000	idle（空闲状态）
0x0001	do_charge_task（充电任务状态）
0x0002	charging（正在充电状态）
0x0003	do_over_charge_task(结束充电状态)
0x0004	pile_error(充电桩断电)
0x0005	safety_warning(请先拔掉充电器)
0x1002	TAGRECETIMEOUT(获取定位信息超时)
0x1003	MARKLAUNCHFAILED(定位算法开启失败)
0x1004	GOTOSTACKFAILED(到达超时)
0x1005	TAGNOVALUE(没有定位信息)
0x1006	TAGPOSEJUMP(定位信息跳变)
0x1007	NOCHARGEPLUG(检测充电电流失败，趴在充电桩)
0x100a	NOCHARGEPLUG_STEP_BACK(检测充电电流失败，退出充电桩)

1.5 地形图模块指令集

地形图模块有以下两种功能：

1. 楼梯落脚点规划：在爬楼梯时，地形图模块会向运动程序输送地形信息，以便规划落脚点，使爬楼梯更稳定。
2. 碰撞检测功能：在非手动模式下，地形图模块会实时检测碰撞风险，对输入的速度（手柄摇杆指令或导航下发的速度指令）进行处理，使机器人减速停下或绕开障碍物，从而避免发生碰撞。

1.5.1 速度输入源

开发者可以自行选择地形图的速度指令输入源：手柄摇杆指令或导航下发的速度指令。

地形图的速度指令输入源设置为手柄摇杆时，称为**辅助模式**，地形图模块会根据碰撞检测的情况对手柄下发的摇杆指令进行处理，实现手动遥控过程中的停避障辅助功能。

地形图的速度指令输入源设置为导航模块时，称为**导航模式**，地形图模块会对导航模块下发的速度指令进行处理，实现导航过程中的安全停障功能。

1.5.1.1 切换速度输入源

在使用地形图的碰撞检测功能时，应先将控制模式切换到非手动模式（参考 1.2.4），然后可向感知主机 **192.168.1.105:43899** 发送如下 UDP 指令，设置需要的速度输入源：

指令名称	指令码	指令值	指令类型
切换速度输入源	0x3101EE03	1=手柄摇杆，2=导航模块	0

【注意】手动模式和非手动模式之间发生切换时并不会重置地形图的速度输入源。

【注意】如果手柄 app 版本低于 v2.3.1，在手柄上从导航模式或辅助模式切回到手动模式时，手柄会自动将地形图的速度输入源重置为手柄摇杆。

1.5.1.2 查看当前速度输入源

感知主机 43899 端口接收到任意来自客户端的消息后，会以 10Hz 的频率反馈地形图碰撞检测功能当前使用的速度输入源：

指令码	指令类型	指令值
0x3100EE03	0	1=手柄摇杆，2=导航模块

1.5.2 地形图模式

在使用地形图功能的过程中，用户需要根据实际环境为地形图设置正确的地形图模式：

1. 在爬楼梯之前，为保证楼梯落脚点规划功能正常工作，需要根据楼梯类型选择正确的地形图模式；
2. 在辅助模式或导航模式下，为使地形图碰撞检测功能正常工作，需要根据环境选择正确的地形图模式。

地形图模式分为四种：实心地面模式、镂空地面模式、无踢面楼梯模式和累积帧模式。

1.5.2.1 设置实心地面模式

实心地面模式适用于机器人落脚平面为实心面的场景，比如普通的实心水泥平地或水泥楼梯。



用户可以向感知主机发送如下 UDP 消息，来切换到实心地面模式：

指令名称	指令码	指令值	指令类型
切换到实心地面模式	0x3101EE01	3	0

【注意】若需要从累积帧模式切换到实心地面模式，需要参考 1.5.2.5 执行从累积帧模式切

出的流程。

1.5.2.2 设置镂空地面模式

镂空地面模式适用于机器人落脚平面为镂空格栅板的场景，比如钢格栅板铺制的工业平台或工业楼梯。



用户可以向感知主机发送如下 UDP 消息，来切换到镂空地面模式：

指令名称	指令码	指令值	指令类型
切换到镂空地面模式	0x3101EE01	4	0

【注意】若需要从累积帧模式切换到镂空地面模式，需要参考 1.5.2.5 执行从累积帧模式切出的流程。

1.5.2.3 设置无踢面楼梯模式

无踢面楼梯模式是实心地面模式的一种特殊工况，适用于机器人落脚面为实心面，但没有垂直踢面的楼梯。



用户可以向感知主机发送如下 UDP 消息，来切换到无踢面楼梯模式：

指令名称	指令码	指令值	指令类型
切换到无踢面楼梯模式	0x3101EE01	5	0

【注意】若需要从累积帧模式切换到无踢面楼梯模式，需要参考 1.5.2.5 执行从累积帧模式切出的流程。

1.5.2.4 设置累积帧模式

累积帧模式适用于落脚平面由多种不同的地面组合而成的场景，例如某段路的地面将由实心地面过渡到镂空地面，或由镂空地面过渡到实心地面，则需使用累积帧模式。累积帧模式不支持地形图碰撞检测功能，且仅支持累积帧模式楼梯步态或累积帧模式 45°楼梯步态。

【注意】累积帧模式（包含累积帧模式相关步态）仅在 vmap（ $\geq 1.5.3$ ）中支持，且需要与 jy_exe（ $\geq 1.10.28$ ）、launcher（ $\geq 0.0.5$ ）、fast-lio（ $\geq 1.0.3$ ）及 jycog-system（ $\geq 0.5.5$ ）配合使用。

设置累积帧模式的步骤如下（示例代码可参考附录 C）：

1. 在设置累积帧模式之前，需要先使机器人保持静止站立，直至成功切换到累积帧模式。
2. 向 **192.168.1.105:60000** 发送如下 UDP 消息，开启激光里程计：

指令码	指令值	指令类型
0xBAA0001	1	0

感知主机在收到上述消息后，会反馈一次执行的结果：

指令码	指令值	指令类型
0xBAA0001	0=成功, -1=失败	0

如果开启失败，需要在执行完从累积帧模式切出的流程（参考 1.5.2.5）之后，才能再次尝试开启激光里程计。

- 开启激光里程计成功后，可以向 **192.168.1.105:43899** 发送如下 UDP 消息，设置累积帧模式：

指令名称	指令码	指令值	指令类型
切换到累积帧模式	0x3101EE01	20	0

- 地形图模块在收到切换到累积帧模式的消息后，会切换到累积帧准备状态，检查激光里程计是否开启成功，并在激光里程计开启后切换到累积帧模式。此时可通过接收 1.5.2.6 中所述的 UDP 消息，判断地形图模式是否已成功切换到累积帧模式。若地形图模块持续反馈处于准备状态超过一定时间（建议超时时间为 10s），则可视其为切换失败，需要在执行完从累积帧模式切出的流程（参考 1.5.2.5）之后，才能从第 1 步开始再次尝试切换到累积帧模式。
- 在确保切换完成后，才可控制机器人开始踏步，继续运动。

1.5.2.5 从累积帧切换到其他模式

在从累积帧模式切换到其他地形图模式时，需要在确保已切换到其他地形图模式之后，向感知主机 **192.168.1.105:60000** 发送如下 UDP 消息，以关闭激光里程计：

指令码	指令值	指令类型
0xBAA0001	0	0

感知主机在收到上述消息后，会反馈一次执行结果：

指令码	指令值	指令类型
0xBAA0001	0=成功, -1=失败	0

1.5.2.6 获取当前地形图模式

感知主机 43899 端口接收到任意来自客户端的消息后, 会以 10Hz 的频率反馈地形图当前所设的地形图模式:

指令码	指令类型	指令值
0x3100EE01	0	3=实心地面 4=镂空地面 5=无踢面楼梯 18=累积帧准备状态 20=累积帧

1.5.3 停避障模式

地形图检测到碰撞风险时, 可以采用停障或避障的模式来避免碰撞。

用户可以向 **192.168.1.105:43899** 发送如下指令, 切换停避障模式:

指令名称	指令码	指令值	指令类型
切换停避障模式	0x3101EE02	1=停障, 2=避障	0

感知主机 43899 端口接收到任意来自客户端的消息后, 会以 10Hz 的频率反馈地形图当前所设的停避障模式:

指令码	指令类型	指令值
0x3100EE02	0	1=停障, 2=避障

1.5.4 障碍物高度阈值

在使用地形图碰撞检测功能的过程中, 地形图模块会将高于障碍物高度阈值的物体视为障碍物。用户可以向 **192.168.1.105:43899** 发送如下指令, 自行设置障碍物高度阈值:

指令码	指令类型	指令值
0x3101EE04	0	1=将高度阈值设置为 8cm, 2=将高度阈值设置为 28cm

需要注意的是，用户应根据所使用的步态对障碍物高度阈值进行调整，建议的阈值如下表所示：

步态	障碍物高度阈值建议值
行走步态或斜坡步态	8cm
越障步态或楼梯步态	28cm

2 感知主机 ROS 话题

本节介绍感知主机（192.168.1.105）中的 ROS 话题。

2.1 传感器驱动话题

该节的话题均为开机后即有消息发布，开发者可订阅话题消息，以获取传感器数据。

话题名	描述	消息类型	频率
/imu/data	本体内部 imu 的数据话题	sensor_msgs/Imu	200Hz
/livox/lidar	览沃激光雷达自定义格式点云数据	livox_ros_driver2/CustomMsg	10Hz
/lidar_points	通用格式点云数据	sensor_msgs/PointCloud2	10Hz

其中话题 `/livox/lidar` 的消息类型 `livox_ros_driver2/CustomMsg` 是自定义消息类型：

```

1  std_msgs/Header header
2      uint32 seq
3      time stamp
4      string frame_id
5  uint64 timebase
6  uint32 point_num
7  uint8 lidar_id
8  uint8[3] rsvd
9  livox_ros_driver2/CustomPoint[] points
10  uint32 offset_time
11  float32 x
12  float32 y
13  float32 z
14  uint8 reflectivity
15  uint8 tag
16  uint8 line

```

使用 `rostopic echo` 命令查看该话题消息前需要执行以下命令获取话题消息类型的定义：

```
1 source ~/jy_cog/transfer/setup.bash
```

开发者如需在自己的程序中订阅该话题，需要在程序中定义该自定义消息的结构。

2.2 基础状态信息话题

该节的话题均为开机后即有消息发布，开发者可订阅话题消息，以获取机器人当前的一些基

基础状态信息。

2.2.1 电池信息

话题名	描述	消息类型
/battery/level	机器人当前剩余电量(%)	std_msgs/UInt8
/battery/current	机器人电流(A 或 10mA)	std_msgs/Float32
/battery/voltage	机器人电压(V)	std_msgs/Float32
/battery_state	机器人电池信息	message_transformer_cpp/BatteryData

其中话题**/battery_state**的消息类型**message_transformer_cpp/BatteryData**是自定义

消息类型：

```
1  uint16 voltage
2  int16 current
3  uint16 remaining_capacity
4  uint16 nominal_capacity
5  uint16 cycles
6  uint16 production_date
7  uint16 balanced_low
8  uint16 balanced_high
9  uint16 protected_state
10 uint8 software_version
11 uint8 battery_level
12 uint8 mos_state
13 uint8 battery_quantity
14 uint8 battery_ntc
15 float32[] battery_temperature
```

部分字段的具体定义如下：

字段名称	字段含义
voltage	电压 (V)
current	电流(A 或 10mA)
remaining_capacity	剩余容量 (10mAh)
nominal_capacity	标称容量 (10mAh)
battery_level	剩余容量百分比 (%)

使用 **rostopic echo** 命令查看该话题消息前需要执行以下命令获取话题消息类型的定义：

```
1 source ~/jy_cog/transfer/setup.bash
```

开发者如需在自己的程序中订阅该话题，需要在程序中定义该自定义消息的结构。

【注意】若运动程序 `jy_exe` 版本低于 `v1.10.19`，则本节介绍的话题中的电流单位为 `A`；若运动程序 `jy_exe` 版本是 `v1.10.19` 或更高版本，则本节介绍的话题中的电流单位为 `10mA`。

2.2.1 运动状态

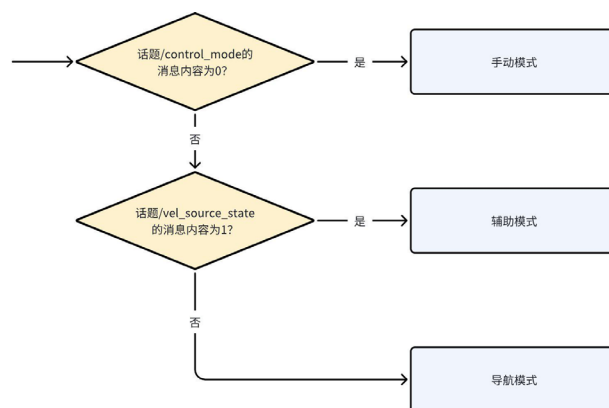
2.2.1.1 控制模式

话题名	描述	消息类型
<code>/control_mode</code>	控制模式	<code>std_msgs/UInt8</code>

话题 `/control_mode` 的消息内容对应的控制模式如下表所示：

话题消息	对应的控制模式
0	手动模式
1	非手动模式

运动程序会根据当前所处的控制模式来决定执行的速度指令的来源：当处于手动模式时，运动程序直接执行手柄发出的原始的速度指令；当处于非手动模式时，运动程序执行地形图修正后的手柄速度指令或导航速度指令。根据地形图速度输入源的不同，非手动模式区分为辅助模式和导航模式，用户可通过话题 `/vel_source_state` 来进行区分（参考 2.4.3）。



2.2.2.2 基本运动状态

话题名	描述	消息类型
<code>/robot_basic_state</code>	机器人基本运动状态	<code>std_msgs/Int32</code>

话题 `/robot_basic_state` 的消息内容对应的基本运动状态如下表所示：

获得的话题消息	对应的基本运动状态
0	趴下状态
1	正在起立状态
2	初始站立状态
3	力控站立状态
4	踏步状态
5	正在趴下状态
6	软急停/摔倒状态
16	RL 状态

2.2.2.3 步态

话题名	描述	消息类型
<code>/robot_gait_state</code>	机器人当前步态	<code>std_msgs/Int32</code>

话题 `/robot_gait_state` 的消息内容对应的步态如下表所示：

获得的话题消息	对应的基本运动状态
0	行走步态
1	越障步态
2	斜坡步态
3	跑步步态
6	楼梯步态（实心地面/镂空地面/无踢面楼梯模式）
7	楼梯步态（累积帧模式）
8	45°楼梯步态（累积帧模式）
32	L 行走步态
33	山地步态
34	静音步态

2.2.2.4 速度

话题名	描述	消息类型
-----	----	------

/robot_velocity	机器人当前速度	geometry_msgs/Twist
-----------------	---------	---------------------

2.2.2.5 运动信息

话题名	描述	消息类型
/leg_odom	世界坐标系下的机器人信息	nav_msgs/Odometry

2.2.2.6 运动时间

话题名	描述	消息类型
/run_time/current_motion_time	当次开机后的运动时间(s)	std_msgs/Int64
/run_time/total_motion_time	累计运动时间(s)	std_msgs/Int64

【注意】若运动程序 jy_exe 版本是 1.10.28 或更早版本，运动时间会在下一次起立后才进行更新。

2.2.3 关节状态

2.2.3.1 关节温度

话题名	描述	消息类型
/driver_temperature	12 个关节的驱动器温度(°C)	std_msgs/UInt8MultiArray
/motor_temperature	12 个关节的电机温度(°C)	std_msgs/Float32MultiArray

12 个关节的排列顺序一般为：左前侧摆关节 fl_hipx、左前髌关节 fl_hipy、左前膝关节 fl_knee、右前侧摆关节 fr_hipx、右前髌关节 fr_hipy、右前膝关节 fr_knee、左后侧摆关节 hl_hipx、左后髌关节 hl_hipy、左后膝关节 hl_knee、右后侧摆关节 hr_hipx、右后髌关节 hr_hipy、右后膝关节 hr_knee。

2.2.3.2 关节数据

话题名	描述	消息类型
/joint_states	关节状态	sensor_msgs/JointState

2.2.4 运行状态

2.2.4.1 运行里程和运行时间

话题名	描述	消息类型
/mileage/current_mileage	当次开机后运行的里程数(cm)	std_msgs/Int32
/mileage/total_mileage	累计运行里程数(cm)	std_msgs/Int32
/run_time/current_run_time	当次开机后的运行时间(s)	std_msgs/Int64
/run_time/total_run_time	累计运行时间(s)	std_msgs/Int64

2.2.4.2 CPU 状态

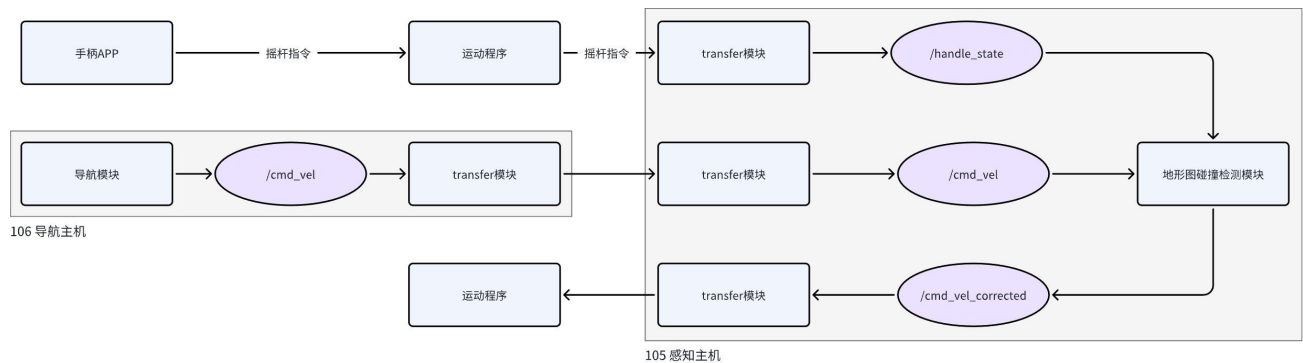
话题名	描述	消息类型
/motion_cpu/frequency	查看机器人 CPU 主频(MHz)	std_msgs/Float32
/motion_cpu/temperature	查看机器人 CPU 温度(°C)	std_msgs/Float32

2.3 速度相关话题

感知主机中包含了三个速度相关的话题：

1. 手柄下发的速度指令 `/handle_state`；
2. 导航模块下发的速度指令 `/cmd_vel`；
3. 经地形图碰撞检测模块处理后的速度指令 `/cmd_vel_corrected`。

在辅助模式或导航模式下，运动程序执行 `/cmd_vel_corrected` 中发布的速度指令（如下图所示）。



2.3.1 手柄摇杆速度指令

该话题在手柄摇杆触发时发布其输入的速度指令。

话题名	描述	消息类型
/handle_state	手柄摇杆速度指令	geometry_msgs/Twist

2.3.2 导航模块速度指令

当导航模块进行路线规划时，该话题会发布导航模块输出的速度指令。

话题名	描述	消息类型
/cmd_vel	导航模块规划的运动速度	geometry_msgs/Twist

2.3.3 地形图处理后的速度指令

当手柄或导航模块的原始速度指令输入后，该话题会发布由地形图碰撞检测模块修正后的速度指令。当没有手柄或导航模块的原始速度指令输入时，开发者也可以根据实际情况直接在该话题中发布消息，使运动程序执行未经地形图碰撞检测模块处理的速度指令。

话题名	描述	消息类型
/cmd_vel_corrected	地形图发布的修正后的速度指令	geometry_msgs/Twist

2.4 地形图相关话题

在非手动模式下，地形图碰撞检测模块会根据局部地形情况，对输入的速度进行修正，运动程序执行修正后的速度，从而避免碰撞。

2.4.1 获取地形图数据

该话题在开机后即有消息发布，开发者可订阅话题消息，以获取地形图数据。

话题名	描述	消息类型
/deeprobotics_local_height_map_mid360/obstacle_pointcloud	身体重力坐标系下的障碍物点云	sensor_msgs/PointCloud2

2.4.2 停避障模式

地形图检测到碰撞风险时，可以采用停障或避障的策略来避免碰撞。

2.4.2.1 设置停避障模式

开发者可通过该话题来设置地形图采用的策略。

话题名	描述	消息类型
/brake_mode	设置停避障模式	std_msgs/Int32

设置不同的停避障模式对应需要发布的话题消息如下表所示：

需要设置的停避障模式	应发布的话题消息
停障	1
避障	2

2.4.2.2 查看停避障模式

该话题在开机后即有消息发布，开发者订阅话题消息，以获取地形图当前的停避障模式。

话题名	描述	消息类型
/brake_mode_state	查看停避障模式	std_msgs/Int32

消息内容对应的停避障模式如下表所示：

获得的话题消息	对应的停避障模式
1	停障
2	避障

2.4.3 速度输入源

开发者可以自行选择地形图的速度指令输入源：手柄摇杆指令或导航下发的速度指令。

地形图的速度指令输入源设置为手柄摇杆时，称为**辅助模式**，地形图模块会根据碰撞检测的情况对手柄下发的摇杆指令进行处理，实现手动遥控过程中的停避障辅助功能。

地形图的速度指令输入源设置为导航模块时，称为**导航模式**，地形图模块会对导航模块下发的速度指令进行处理，实现导航过程中的安全停障功能。

2.4.3.1 切换速度输入源

在使用地形图的碰撞检测功能时，应先将控制模式切换到非手动模式，然后可以向话题

`/vel_source` 中发布消息，设置需要的速度输入源。

话题名	描述	消息类型
<code>/vel_source</code>	设置速度输入源	<code>std_msgs/Int32</code>

设置不同的速度输入源对应需要发布的话题消息如下表所示：

需要设置的速度输入源	应发布的话题消息
手柄摇杆	1
导航模块	2

【注意】手动模式和非手动模式之间发生切换时并不会重置地形图的速度输入源。

【注意】如果手柄 app 版本低于 2.3.1，在手柄上从导航模式或辅助模式切回到手动模式时，手柄会自动将地形图的速度输入源重置为手柄摇杆。

2.4.3.2 查看当前速度输入源

该话题在开机后即有消息发布，开发者可订阅话题消息，以获取地形图当前使用的速度输入源。

话题名	描述	消息类型
<code>/vel_source_state</code>	查看当前速度输入源	<code>std_msgs/Int32</code>

消息内容对应的速度输入源如下表所示：

获得的话题消息	对应的速度输入源
1	手柄摇杆
2	导航模块

2.4.4 地形图模式

使用地形图功能的过程中，用户需要根据实际环境为地形图设置正确的地形图模式：

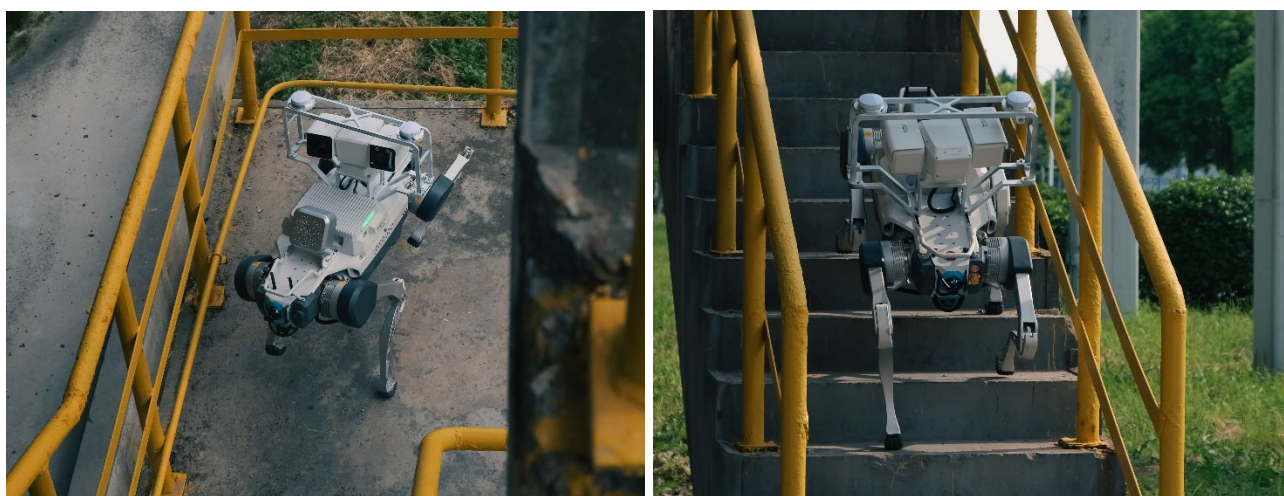
1. 在爬楼梯之前，为保证楼梯落脚点规划功能正常工作，需要根据楼梯类型选择地形图模式；

2. 在辅助模式或导航模式下，为使地形图碰撞检测功能正常工作，需要根据环境选择正确的地形图模式。

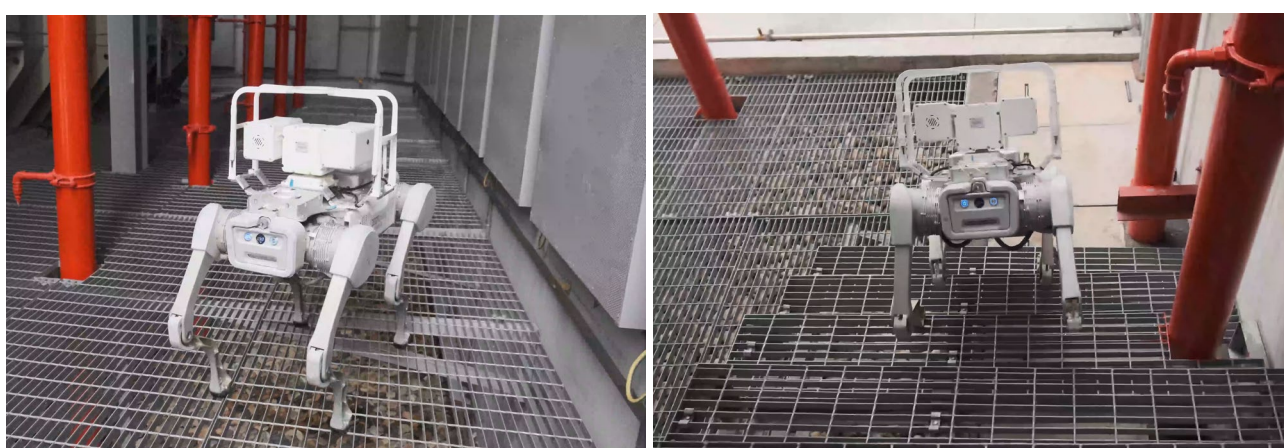
地形图模式分为四种：实心地面模式、镂空地面模式、无踢面楼梯模式和累积帧模式。

2.4.4.1 设置实心地面/镂空地面/无踢面楼梯模式

实心地面模式适用于机器人落脚的平面为实心面的场景，比如普通的实心水泥平地或水泥楼梯。



镂空地面模式适用于机器人落脚的平面为镂空格栅板的场景，比如钢格栅板铺制的工业平台或工业楼梯。



无踢面楼梯模式是实心地面模式的一种特殊工况，适用于机器人落脚面为实心面，但没有垂直踢面的楼梯。



在进入落脚面类型不同的环境之前，开发者可通过该话题来为地形图设置合适的模式。

话题名	描述	消息类型
/height_map_mode	设置地形图模式	std_msgs/Int32

设置不同的地形图模式对应需要发布的话题消息如下表所示：

需要设置的地形图模式	应发布的话题消息
实心地面	3
镂空地面	4
无踢面楼梯	5

2.4.4.2 设置累积帧模式

累积帧模式适用于落脚平面由多种不同的地面组合而成的场景，例如某段路的地面将由实心地面过渡到镂空地面，或由镂空地面过渡到实心地面，则需标为累积帧模式。累积帧模式不支持地形图碰撞检测功能，且仅支持累积帧模式楼梯步态或累积帧模式 45°楼梯步态。

【注意】累积帧模式（包含累积帧模式相关步态及累积帧模式）仅在 vmap（ $\geq 1.5.3$ ）中支持，且需要与 jy_exe（ $\geq 1.10.28$ ）、launcher（ $\geq 0.0.5$ ）、fast-lio（ $\geq 1.0.3$ ）及 jycog-system（ $\geq 0.5.5$ ）配合使用。

设置累积帧模式的步骤如下（流程图可参考附录 C）：

1. 在设置累积帧模式之前，需要先使机器人保持静止站立，直至成功切换到累积帧模式。
2. 向 **192.168.1.105:60000** 发送如下 UDP 消息（格式参考 1.1），以开启激光里程计：

指令码	指令值	指令类型
0xBAA0001	1	0

感知主机在收到上述消息后，会反馈一次执行的结果：

指令码	指令值	指令类型
0xBAA0001	0=成功，-1=失败	0

如果执行失败，需要在执行完从累积帧模式切出的流程（参考 2.4.4.3）之后，才能再次尝试开启激光里程计。

3. 开启激光里程计后，向话题 `/height_map_mode` 发布消息，切换到累积帧模式：

话题名	描述	消息类型
/height_map_mode	设置地形图模式	std_msgs/Int32

设置累积帧模式需要发布的话题消息如下表所示：

需要设置的地形图模式	应发布的话题消息
累积帧模式	20

4. 地形图模块在收到切换到累积帧模式的消息后，会先切换到累积帧准备状态，检查激光里程计是否开启成功，在确认激光里程计开启后，才会切换到累积帧模式。开发者可参考 2.4.4.4 判断地形图模式是否已成功切换到累积帧模式。若地形图模块持续反馈处于准备状态超过一定时间（建议超时时间为 10s），则可视其为切换失败，需要在执行完从累积帧模式切出的流程（参考 2.4.4.3）之后，才能从第 1 步开始再次尝试切换到累积帧模式。
5. 在确保切换完成后，才可控制机器人开始踏步，继续运动。

2.4.4.3 从累积帧切换到其他类型

在从累积帧模式切换到其他地形图模式时，需要在确保已切换到其他地形图模式之后，向感知主机 **192.168.1.105:60000** 发送如下 UDP 消息（格式参考 1.1），以关闭激光里程计：

指令码	指令值	指令类型
0xBAA0001	0	0

感知主机在收到上述消息后，会反馈一次执行的结果：

指令码	指令值	指令类型
0xBAA0001	0=成功，-1=失败	0

2.4.4.4 查看地形图模式

该话题在开机后即有消息发布，开发者订阅话题消息，以获取地形图当前所设的地形图模式。

话题名	描述	消息类型
/height_map_mode_state	查看地形图模式	std_msgs/Int32

消息内容对应的地形图模式如下表所示：

获得的话题消息	对应的地形图模式
3	实心地面
4	镂空地面
5	无踢面楼梯
18	累积帧准备状态
20	累积帧

2.4.5 障碍物高度阈值

地形图碰撞检测模块会将超过障碍物高度阈值的物体视为障碍物进行停避障。

2.4.5.1 设置障碍物高度阈值

开发者应当根据不同步态跨越障碍的能力，针对性地设置不同的障碍物高度阈值。一般情况下，楼梯步态设置为 0.25，斜坡步态设置为 0.12，行走步态设置为 0.08。由于会将草丛视为

障碍物，过草地时可按实际情况适当提高阈值。

话题名	描述	消息类型
/step_z_max	设置障碍物高度阈值（米）	std_msgs/Float64

2.4.5.2 查看障碍物高度阈值

该话题在开机后即有消息发布，开发者订阅话题消息，以获取地形图当前所设的障碍物高度阈值。

话题名	描述	消息类型
/step_z_max_state	查看障碍物高度阈值（米）	std_msgs/Float64

2.4.6 触发修正的碰撞时间

当地形图预测到将在设置的时间阈值内发生碰撞，就会对原始的速度指令进行修正。

2.4.6.1 设置触发修正的碰撞时间

开发者可通过该话题来修改时间阈值。

话题名	描述	消息类型
/slow_t	设置触发修正的碰撞时间（秒）	std_msgs/Float64

2.4.6.2 查看触发修正的碰撞时间

该话题在开机后即有消息发布，开发者订阅话题消息，以获取地形图当前所设的触发修正的碰撞时间阈值。

话题名	描述	消息类型
/slow_t_state	查看触发修正的碰撞时间（秒）	std_msgs/Float64

2.4.7 触发期望速度置零的碰撞时间

当地形图预测到将在设置的时间阈值内发生碰撞，就会直接将原始速度修正为 0。

2.4.7.1 设置触发期望速度置零的碰撞时间

开发者可通过该话题来修改时间阈值。

话题名	描述	消息类型
/stop_t	设置触发期望速度置零的碰撞时间（秒）	std_msgs/Float64

2.4.7.2 查看触发期望速度置零的碰撞时间

该话题在开机后即有消息发布，开发者订阅话题消息，以获取地形图当前所设的触发期望速度置零的碰撞时间阈值。

话题名	描述	消息类型
/stop_t_state	查看触发期望速度置零的碰撞时间（秒）	std_msgs/Float64

2.5 自主充电相关话题

该节的话题均为开机后即有消息发布，开发者可订阅话题消息，以获取自主充电状态。

话题名	描述	消息类型
/charge_manager_state	获取自主充电状态	std_msgs/Int32

话题 `/charge_manager_state` 的消息内容对应的自主充电状态如下表所示：

获得的话题消息	对应的自主充电状态
0	空闲
1	正在做充电任务
2	正在充电
3	正在做结束充电任务
4	充电桩断电
4098	获取定位信息超时
4099	定位算法开启失败
4100	到达超时
4101	没有定位信息
4102	定位信息跳变
4103	检测充电电流失败，趴在充电桩

4106

检测充电电流失败，退出充电桩

3 导航主机 ROS 话题

本节介绍导航主机（192.168.1.106）中的 ROS 话题。

3.1 传感器驱动话题

导航主机的传感器驱动话题与感知主机一致，请参考 2.1 节内容。

3.2 基础状态信息话题

导航主机的基础状态信息话题与感知主机一致，请参考 2.2 节内容。

3.3 定位相关话题

3.3.1 重置定位

该话题在定位程序启动后激活，开发者可在该话题中发布消息，以重置机器人定位。

话题名	描述	消息类型
/initialpose	为定位重置当前的位姿	geometry_msgs/PoseWithCovarianceStamped

3.3.2 切换全局地图

该话题在定位程序启动后激活，开发者可在该话题中发布消息，为定位设置新的全局地图。

话题名	描述	消息类型
/map_request/pcd	为定位设置的新全局地图（.pcd 文件）的路径	std_msgs/String

3.3.3 获取实时位姿

该话题在定位程序启动后即有消息发布，开发者可订阅该话题，以获取机器人实时位姿。

话题名	描述	消息类型
/odom	定位给出的机器人实时位姿	nav_msgs/Odometry

3.3.4 获取全局点云地图

该话题在定位程序启动后即有消息发布，开发者可订阅该话题，以获取全局点云地图。

话题名	描述	消息类型
/globalmap	定位中使用的全局点云地图	sensor_msgs/PointCloud2

3.3.5 获取定位丢失状态

该话题在定位程序启动后即有消息发布，开发者可订阅该话题，以查看定位是否丢失：

话题名	描述	消息类型
/location_status	定位丢失状态	std_msgs/Int8

话题 `/location_status` 的消息内容对应的定位丢失状态如下表所示：

获得的话题消息	对应的定位丢失状态
0	定位正常
1	定位丢失

3.4 导航相关话题

3.4.1 获取导航中使用的全局栅格地图

该话题在导航程序启动后即有消息发布，开发者可订阅该话题，以获取导航中使用的全局栅格地图：

话题名	描述	消息类型
/map	导航中使用的全局栅格地图	nav_msgs/OccupancyGrid

3.4.2 下发普通导航目标点

该话题在导航程序启动后激活，开发者可以在该话题中发布消息，以指定普通导航目标点：

话题名	描述	消息类型
/move_base/goal	普通导航目标点	MoveBaseActionGoal

其中消息类型 `MoveBaseActionGoal` 是自定义消息类型：

```

1  std_msgs/Header header
2      uint32 seq
3      time stamp

```

```

4   string frame_id
5   actionlib_msgs/GoalID goal_id
6   time stamp
7   string id
8   move_base_msgs/MoveBaseGoal goal
9   geometry_msgs/PoseStamped target_pose
10  std_msgs/Header header
11  uint32 seq
12  time stamp
13  string frame_id
14  geometry_msgs/Pose pose
15  geometry_msgs/Point position
16  float64 x
17  float64 y
18  float64 z
19  geometry_msgs/Quaternion orientation
20  float64 x
21  float64 y
22  float64 z
23  float64 w

```

使用 `rostopic echo` 命令查看该话题消息前需要执行以下命令获取话题消息类型的定义：

```
1 | source ~/jy_cog/broker/setup.bash
```

若上述命令无效，请执行以下命令再次尝试获取话题消息类型的定义：

```
1 | source ~/jy_cog/nav/setup.bash
```

开发者如需在自己的程序中订阅该话题，需要在程序中定义该自定义消息的结构。

3.4.3 下发直线导航目标点

该话题在导航程序启动后激活，开发者可以在该话题中发布消息，以指定直线导航目标点：

话题名	描述	消息类型
/move_base_direct_line/goal	直线导航目标点	MoveBaseDirectLineActionGoal

其中消息类型 `MoveBaseDirectLineActionGoal` 是自定义消息类型：

```

1   std_msgs/Header header
2   uint32 seq
3   time stamp
4   string frame_id
5   actionlib_msgs/GoalID goal_id
6   time stamp
7   string id

```

```

8  move_base_msgs/MoveBaseDirectLineGoal goal
9      geometry_msgs/PoseStamped target_pose
10     std_msgs/Header header
11         uint32 seq
12         time stamp
13         string frame_id
14     geometry_msgs/Pose pose
15         geometry_msgs/Point position
16             float64 x
17             float64 y
18             float64 z
19         geometry_msgs/Quaternion orientation
20             float64 x
21             float64 y
22             float64 z
23             float64 w
24     bool is_backward_mode

```

使用 `rostopic echo` 命令查看该话题消息前需要执行以下命令获取话题消息类型的定义：

```
1 source ~/jy_cog/broker/setup.bash
```

若上述命令无效，请执行以下命令再次尝试获取话题消息类型的定义：

```
1 source ~/jy_cog/nav/setup.bash
```

开发者如需在自己的程序中订阅该话题，需要在程序中定义该自定义消息的结构。

3.4.4 获取普通导航规划出的全局路径

当开发者下发普通导航目标点后，可以通过订阅该话题获取普通导航规划出的全局路径：

话题名	描述	消息类型
/move_base/GlobalPlanner/plan	普通导航规划的全局路径	nav_msgs/Path

3.4.5 获取普通导航规划出的局部路径

当开发者下发普通导航目标点后，可以通过订阅该话题获取普通导航规划出的局部路径：

话题名	描述	消息类型
/move_base/LocalPlanner/local_plan	普通导航规划的局部路径	nav_msgs/Path

3.4.6 获取普通导航局部规划器截取的全局路径

当开发者下发普通导航目标点后，可以通过订阅该话题获取普通导航局部规划器截取的全局

路径：

话题名	描述	消息类型
/move_base/LocalPlanner/global_plan	普通导航局部规划器截取的全局路径	nav_msgs/Path

3.4.7 获取导航停避障状态

当机器狗正在导航时，开发者可以订阅该话题，获取导航停避障状态：

话题名	描述	消息类型
/move_base/obs_state	停避障状态	std_msgs/Int32

话题 `/move_base/obs_state` 的消息内容对应的停避障状态如下表所示：

获得的话题消息	对应的停避障状态
0	未检测到碰撞风险
1	正在停避障

3.4.8 获取或下发导航速度指令

当开发者下发导航目标点后，可以通过订阅该话题获取导航规划得到的速度指令：

话题名	描述	消息类型
/cmd_vel	导航规划得到的速度指令	geometry_msgs/Twist

开发者也可以在本话题中下发自行规划的速度指令（各步态的最大速度详见附录 B），在导航模式下，该话题消息会被转发给感知主机中的地形图模块进行处理，详见 2.3。

附录 A UDP 传输数据内容详解

本节以机器人关节角度上传数据为例进行分析。

```

1  /* 原始数据 总长度 108 个字节
2  * 0209 0000 6000 0000 0100 0000 2efd 3ccf
3  * 47e4 e1bf f130 4992 f49e e7bf c3d7 eeef
4  * 8567 f13f 9ae4 b66d 6583 f1bf 24d8 f33c
5  * 3d00 f13f 8c16 222a 0b1f 0140 e8ba aaaa
6  * 3803 ef3f d009 0000 60bd d2bf db04 3353
7  * 1150 e73f 7d0b 0000 72f0 e53f 0a3d 0cc3
8  * f071 e73f c3d7 eeef 7b52 f93f
9  */
10 /******
11 /// 内容解析
12 //0209 0000      //xxxx 命令字为 0x0902
13 //6000 0000      //yyyy 正文数据长度为 0x60 即长度为 96 个字节
14 //0100 0000      //zzzz 为 1，表示复杂指令，后方有正文数据
15 /*
16 * bbbbbbbbbb.....
17 * 以下为数据正文内容，共计 96 个字节，数据为长度为 12 的 double 类型的数组
18 * 2efd 3ccf 47e4 e1bf f130 4992 f49e e7bf
19 * c3d7 eeef 8567 f13f 9ae4 b66d 6583 f1bf
20 * 24d8 f33c 3d00 f13f 8c16 222a 0b1f 0140
21 * e8ba aaaa 3803 ef3f d009 0000 60bd d2bf
22 * db04 3353 1150 e73f 7d0b 0000 72f0 e53f
23 * 0a3d 0cc3 f071 e73f c3d7 eeef 7b52 f93f
24 */
25 //即对应机器人关节角度
26 double joint_angle[12] = {-0.5591162726995316, -0.7381537301203293, 1.0877742727584383,
27                             -1.0945791516990426, 1.0625584011991203, 2.1401580135014395,
28                             0.9691432317102597, -0.2928085327149832, 0.7285238862024487,
29                             0.6856012344363617, 0.7326587495353192, 1.5826377828902742};

```

附录 B 各步态最大速度限制

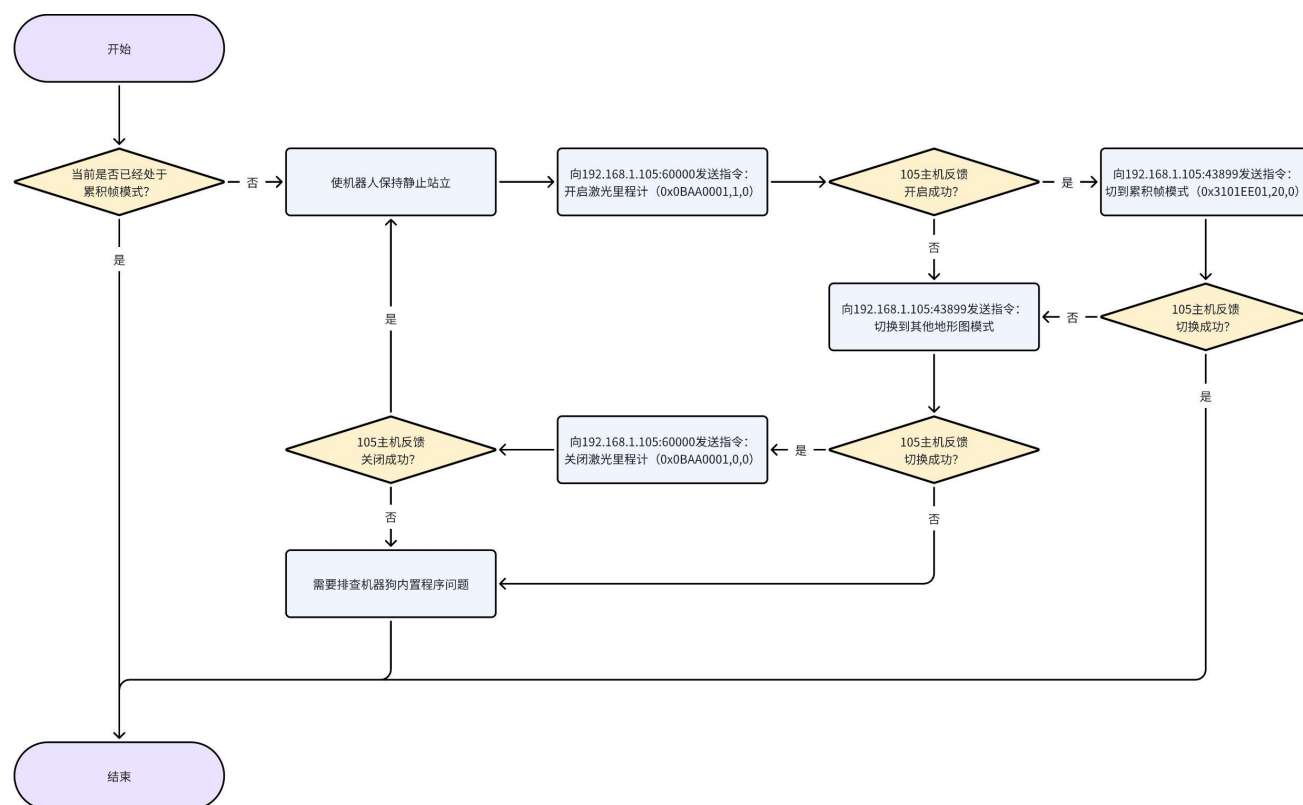
机器人处于正常的身体高度状态时，各个步态的最大速度限制如下表所示：

步态（正常高度下）	最大前进速度	最大后退速度	最大侧向速度	最大转向速度
行走步态	1.2 m/s	0.75 m/s	0.15 m/s	0.45 rad/s
斜坡步态	0.7 m/s	0.75 m/s	0.25 m/s	0.5 rad/s
越障步态	0.3 m/s	0.4 m/s	0.1 m/s	0.5 rad/s
楼梯步态（实心地面模式）	0.3 m/s	0.3 m/s	0.2 m/s	0.8 rad/s
楼梯步态（镂空地面模式）	0.3 m/s	0.3 m/s	0.2 m/s	0.8 rad/s
楼梯步态（无踢面楼梯模式）	0.3 m/s	0.3 m/s	0.2 m/s	0.8 rad/s
楼梯步态（累积帧模式）	0.6 m/s	0.3 m/s	0.2 m/s	0.8 rad/s
45°楼梯步态（累积帧模式）	0.3 m/s	0.2 m/s	0.2 m/s	0.8 rad/s
L 行走步态	1.0 m/s	1.0 m/s	0.5 m/s	1.2 rad/s
山地步态	1.0 m/s	1.0 m/s	0.8 m/s	1.2 rad/s

机器人处于匍匐高度时，行走步态和斜坡步态的最大前进速度降低至原来的 50%。

附录 C UDP 示例代码（累积帧模式的使用）

根据 1.5.2.4 所述，开启累积帧模式的基本流程如下：



1. 开启和关闭激光里程计的示例代码如下：

```

1 // 协议格式：UDP（请求应答式：发送一次请求，反馈一次应答）
2 // 默认 IP: 192.168.1.105    PORT: 60000
3
4 /*开启激光里程计时，需要机器狗完全站立，保证姿态稳定*/
5
6 struct SimpleCommander {
7     uint32_t command;
8     uint32_t value; // 0 关闭  1 开启
9     uint32_t type;
10 };
11
12 int8_t Dispatch(int value) {
13     SimpleCommander commander;
14     commander.command = 0x0BAA0001;
15     commander.value = value;
16     commander.type = 0;
  
```

```

17
18 auto ret = sendto(socket_fd_, (void*)&commander, 12, 0,
19                  (struct sockaddr *)&addr_, sizeof(addr_));
20 SimpleCommander recv_commander;
21 if(ret > 0) {
22     socklen_t addr_len = sizeof(addr_);
23     ret = recvfrom(socket_fd_, &recv_commander, sizeof(SimpleCommander), 0,
24                  (struct sockaddr *)&addr_, &addr_len);
25     // recv_commander.value    0 开启/关闭 成功   -1 开启/关闭 失败
26     if(ret <= 0) return -1;
27 } else return -1;
28 return 0;
29 }

```

2. 获取当前地形图模式的示例代码如下：

```

1 // 协议格式：UDP（触发式：接收到第一次请求后，会持续反馈相关状态）
2 // 默认 IP：192.168.1.105    PORT：43899
3
4 struct SimpleCommander {
5     uint32_t command;
6     uint32_t value;
7     uint32_t type;
8 };
9
10 int8_t GetVMapData(int value) {
11     SimpleCommander commander;
12     commander.command = 0x3101EEFF;
13     commander.value = 0;
14     commander.type = 0;
15
16     auto ret = sendto(socket_fd_, (void*)&commander, 12, 0,
17                     (struct sockaddr *)&addr_, sizeof(addr_));
18     SimpleCommander recv_commander;
19     if(ret > 0) {
20         socklen_t addr_len = sizeof(addr_);
21         while(true) {
22             ret = recvfrom(socket_fd_, &recv_commander, sizeof(SimpleCommander), 0,
23                         (struct sockaddr *)&addr_, &addr_len);
24             if(ret > 0) {
25                 if(recv_commander.command == 0x3100EE01) {
26                     // 地形图当前的模式 = recv_commander.value;
27                     // 3 实心地面 4 镂空地面 5 无踢面楼梯 18 累积帧准备状态 20 累积帧模式
28                 }
29             }

```



```
30     }
31     } else return -1;
32     return 0;
33 }
```

3. 设置地形图模式的示例代码如下：

```
1  // 协议格式：UDP（触发式）
2  // 默认 IP：192.168.1.105    PORT：43899
3
4  /*指令发送后，要等待是否切换到对应的地形图模式*/
5  /*通过 上述 2 持续获取当前的地形图模式*/
6  /* 2 和 3 使用同一个端口 */
7
8  struct SimpleCommander {
9      uint32_t command;
10     uint32_t value; // 3 设置实心地面    4 设置镂空地面    5 设置无踢面楼梯    20 设置累积帧模式
11     uint32_t type;
12 };
13
14 int8_t Dispatch(int value) {
15     SimpleCommander commander;
16     commander.command = 0x3101EE01;
17     commander.value = value;
18     commander.type = 0;
19
20     auto ret = sendto(socket_fd_, (void*)&commander, 12, 0,
21                       (struct sockaddr *)&addr_, sizeof(addr_));
22     return 0;
23 }
```